Operating System

Lecture-1
19.07.2026

Operation System (OS) is a system software, that operates the system.

→ Interface between user and hardware

→ Memory Management.

→ cpu scheduling

Examples · Mac OS, Linux, & Windows, Android, IOS.

Q. What is an Operating System?

Lecture-2
19.07.2026

⇒ An Operating System (OS) is a ~~gro~~ program that acts as an intermediary between a computer user and the computer hardware.

⇒ Os is software that controls the operation of computer and directs the processing of programs.

Goal: i) Execute user program

ii) Make the computer system convenient and user-friendly to use.

iii) Use computer hardware in an effieient manner.

⇒ Computer System Structure: There are 4 main components:

① Hardware: Provides basic computing resources.
Example: CPU, Memory, I/O devices.

② Operating System: Controls and coordinates use of hardware among various applications and users.
Example: MacOS, Windows, Linux

③ Application Program: Define how system resources are used to solve users computing problems.
Example: Compilers, web browsers, video games etc.

④ Users: Entities interacting with the system.
Example: People, machines, other components.

System Perspective

What an OS does depends on the user's point of view:

2 point of views
- user view
  - ease of use
  - good performance
- system view
  - efficient resource allocation
  - control program

⇒ Single-User System: Prioritize convenience, ease of use and good performance; resource utilization is not primary concern.

2. Shared Computers: focus on maximizing resource sharing so all concurrent users get fair access.

3. Workstations: Balance individual usability with sharing centralized resources from network servers.

4 Handheld/Mobile Devices: Focus on battery conservation and user responsiveness under limited hardware resources.

5. Embedded System: Found inside cars, appliances or machinery. Focused entirely on background operations.

Formal Definitions of OS functions

Resource Allocator: Manages system resources and resolves competing demands fairly and efficiently.

Control Program: Controls execution of programs to prevent errors and improper use of the computer

Kernel: The primary Software components that remains loaded and actively running in memory at all times

Middleware: A set of software framworks that provide additional services to application developers.

Bootstrap / Bootloader

Startup Role: A small initialization program loaded automatically when a computer powers up or reboots.

Storage Location: Stored permanently in non-volatile memory chips (ROM or EPROM), commonly known as firmware.

Task Executed: 1. Initializes hardware components across the board

2. Locates and loads the OS kernel into main memory to launch the OS.

Steps: :⇒ start up operation

⇒ CPU on → ROM এর কাছে Instruction থাকে।

⇒ ROM এর মধ্যে software হিসেবে থাকে firmware

⇒ Firmware এর মধ্যে থাকে BIOS (Basic Input Output System)

⇒ BIOS এর code থাকে এবং initialize হলে active হবে।

⇒ OS secondary memory তে থাকে এবং H.D. থেকে OS এর copy RAM-এ আসে।

Computer System Organization

Hardware Interconnection: One or more CPUs together with various device controllers connect a single common bus providing access to shared memory.

Concurrent Execution: Hardware devices and processors operate at the same time, competing for available memory access cycle.

Types of Operating System

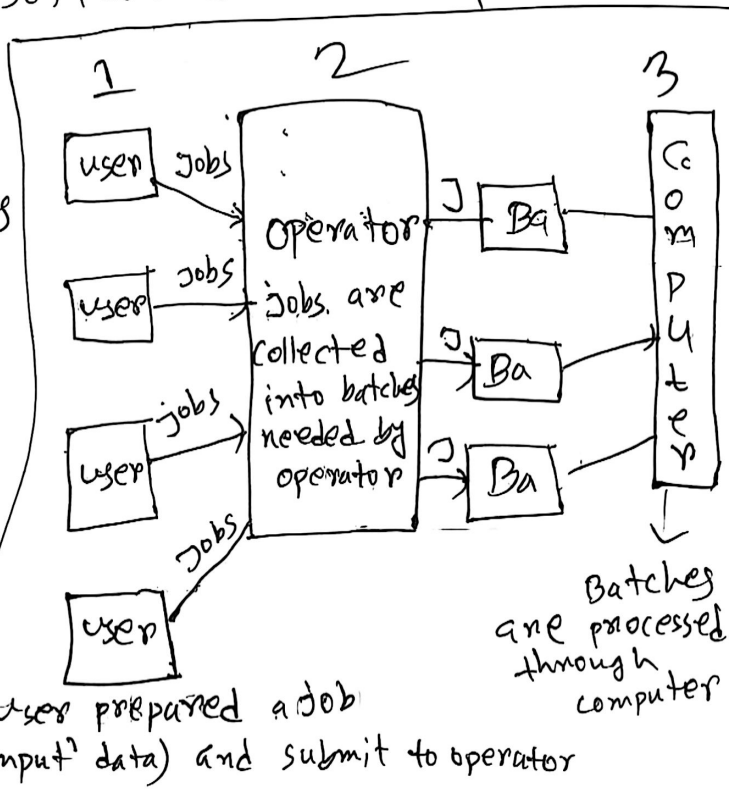
Lecture-3
26.07.2026

Batch Operating System: A system designed for non-interactive tasks where jobs are executed automatically without direct user intervation

Batching Concept: To speedup overall execution, individual user jobs requiring similar software or setup are groped together into batches.

How it works: The system holds queued jobs in memory and processes them sequentially in First-Come, First-Served (FCFS) order.

When a job is completed,



its allocated memory is freed and its output is redi-
-rected to an output spool for later printing.

Examples: (i) Telecom Billing: Batch processing monthly telephone call records to generate customer bills.

(ii) Bank Processing: Running overnight processing for inter-national transfers or clearing accounts.

(iii) Corporate Payroll Systems: Calculating monthly salaries and printing employee pay slips.

Advantages: (i) Faster Group Execution: Same jobs in the batch are higher executed speed.

(ii) No user Overhead: Automated transitions mean the next job running instantly once the active job finishes.

(iii) Better CPU Utilization: Minimizes CPU idle time.

(iv) Task Partitioning: Large batch processes are divided into smaller sub-processes for better handling.

Disadvantages: (i) Difficult to debug

(ii) Infinite loop risk

(iii) Batch systems are costly.

Time-Sharing Operating System

Def:
A system model that allows multiple users at different terminals to access a single central computer at the same time.

Time Quantization: The CPU processing assigns an equal portion of time (called a time ~~quantum~~ slices) to each task so every process run smoothly.

U1 (10 min) / 7 min / 4 min / 1 min / x
U2 (15 min) / 12 min / 9 min / 6 min / 3 min / x
U3 (3 min) / 0 min / x

TQ = 3 min

Characteristics: (i) Multi-user system
(ii) Fixed time (iii) Fair CPU sharing

Multitasking: Each user receives a portion of CPU time making these systems effectively known as Multi-tasking System.

Time-sharing is considered a logical step forward from basic ~~mut~~ multiprogramming.

Examples: Multics, Unix etc.

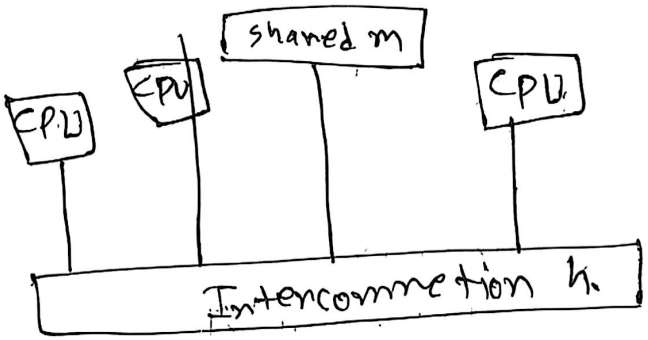
Distributed Operating System

Definition: A system whose individual components are spread across differential networked computers that communi-cates by passing meassage. Example: Locus, AEGIS, etc

⇒ Interconnected autonomous computers run together, where each individual system has its own CPU and memory unit. This is called loosely coupled systems. (Fig: slide-23)

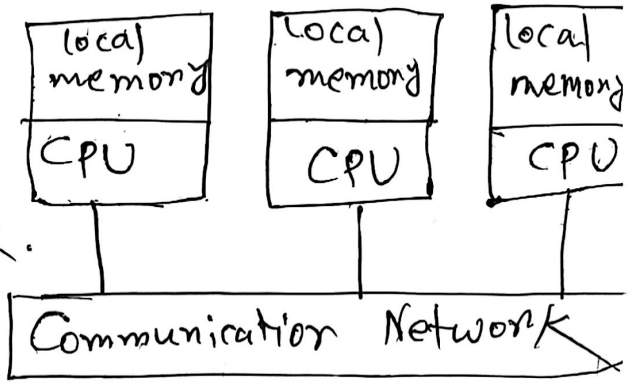
Coupled Systems

Tightly Coupled System: Processors share a single main memory and communicate directly over a high-speed bus. Examples: Parallel Processing, shared memory multiprocessors



Loosely Coupled Systems:

Independent computers have separate memory units and exchange messages over a network.



Example:

Network Operating Systems (NOS)

Definition: An operating system running on a network host that enables shared access to files, printers, security services and applications across a private network.

Examples: Windows Server 2003/2008, UNIX, Linux, macOS. X, Novell NetWare, BSD.

Distributed OS vs Network OS

Feature	Distributed OS	Network OS
Definition	Multiple computers work together as one system	Multiple independent computers connected by a network
User view	Appears as a single Computer	Users know they are using different computers
System Image	Single System Image (SSI)	No single system image
Transparency	High: location of resources is hidden.	Low; users explicitly access remote resources
Operating System	Cooperating distributed OS across nodes	Each computer runs its own OS independently
Scalability	Excellent	Good, but manual adminstration needed
Adminstration	Appears Centralized	Each machine is ~~mine~~ monitored separately
Examples	Amoeba, Plan 9, Sprite	Windows Server, Ubuntu Server, macOS X.

Mobile Operating System

Target Hardware: Designed specifically for handheld smartphones and tablets

Special features: Includes built-in support for hardware sensors like GPS and gyroscopes, allowing advanced apps such as augmented reality.

Connectivity: Connects using cellular data networks or IEEE 802.11 wireless (WiFi)

Market leaders: Apple iOS, Groove Android.

Embedded Operating System

Definition: Specialized operating systems designed for computers buitinto larger machinery or consumer appliance

Application: Vechicles, traffic signals, smart TVs, ATMs, flight controls, Point of Sale (POS), GPS receivers, digital cameras.

Real-Time Operating System (RTOS)

Definition: An operating system designed to gurantee that specific tasks complete within strict, predefined time limits.

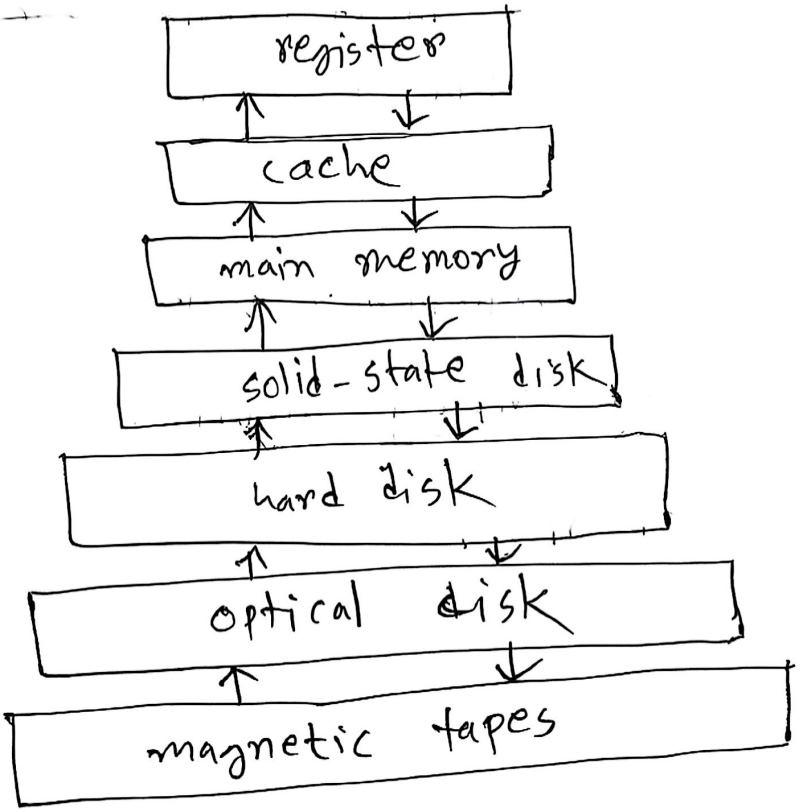
Core Requirement: Operations must be produce correct logic within strict deadline constraints.

Applications: Industrial machinery, medical imaging, flight control, weapon control system, scientific experiments.

Lecture-4
28.07.2026

Storage-Device

page-30, 31

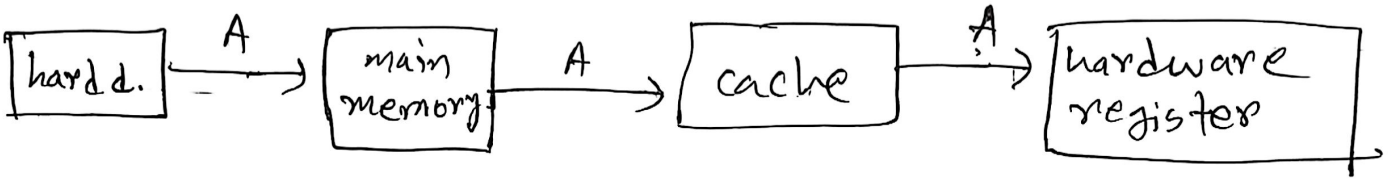


Caching

Concept: Data currently in use is temporarily copied from slower main storage to faster memory to speed up overall execution.

Migration of Data 'A' from Disk to Register

Multitasking environments must be careful to use most recent value, no matter where it is stored in the storage hierarchy.



Multiprocessor environment must provide cache coherency in hardware such that all CPUs have the most recent value in their cache.

Cache Coherency: Cache coherency helps to prevent cache inconsistancy. MESI algorithm is an example used in cache coherency.

Computer-System Architecture

Most system use a single general-purpose processor.

Single vs Multiprocessor: Most legacy computers use one main CPU, but multiprocessor also known as parallel system or tightly coupled systems are widely used today.

Key Advantages: ① Increased Throughput: Accomplishes more work in less time.

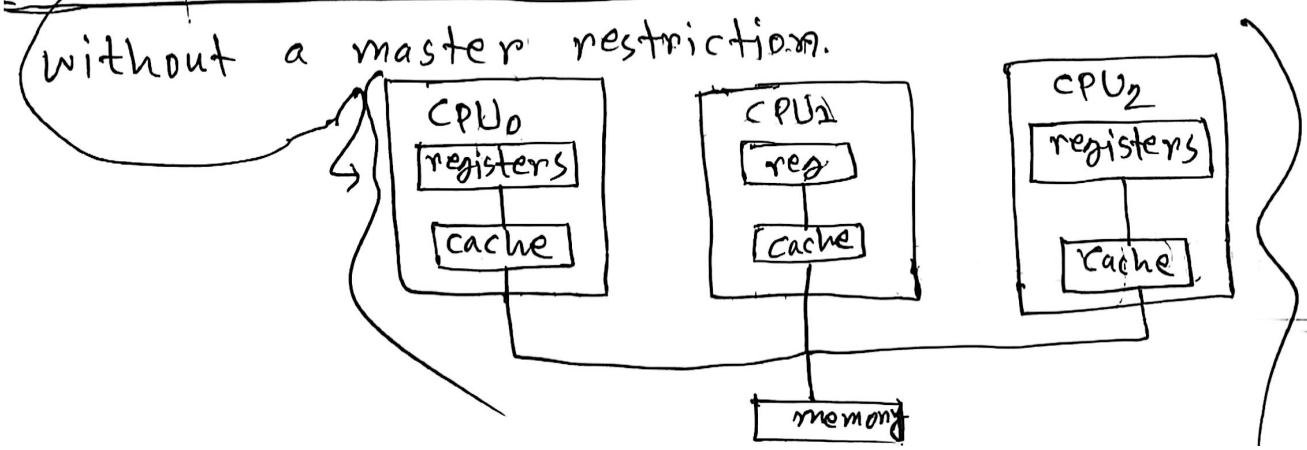
~~work~~

② Economy of Scale: Sharing hardware resources costs less than separate standalone computers.

③ Increased Reliability: If one processor fails, the system continues running at a reduced pace (graceful degradation).

Two types: Asymmetric Multiprocessing: Each processor is assigned a specific task.

Symmetric Multiprocessing: Every processor performs all tasks without a master restriction.



page-36, 37 from slide

Dual-Mode Operation

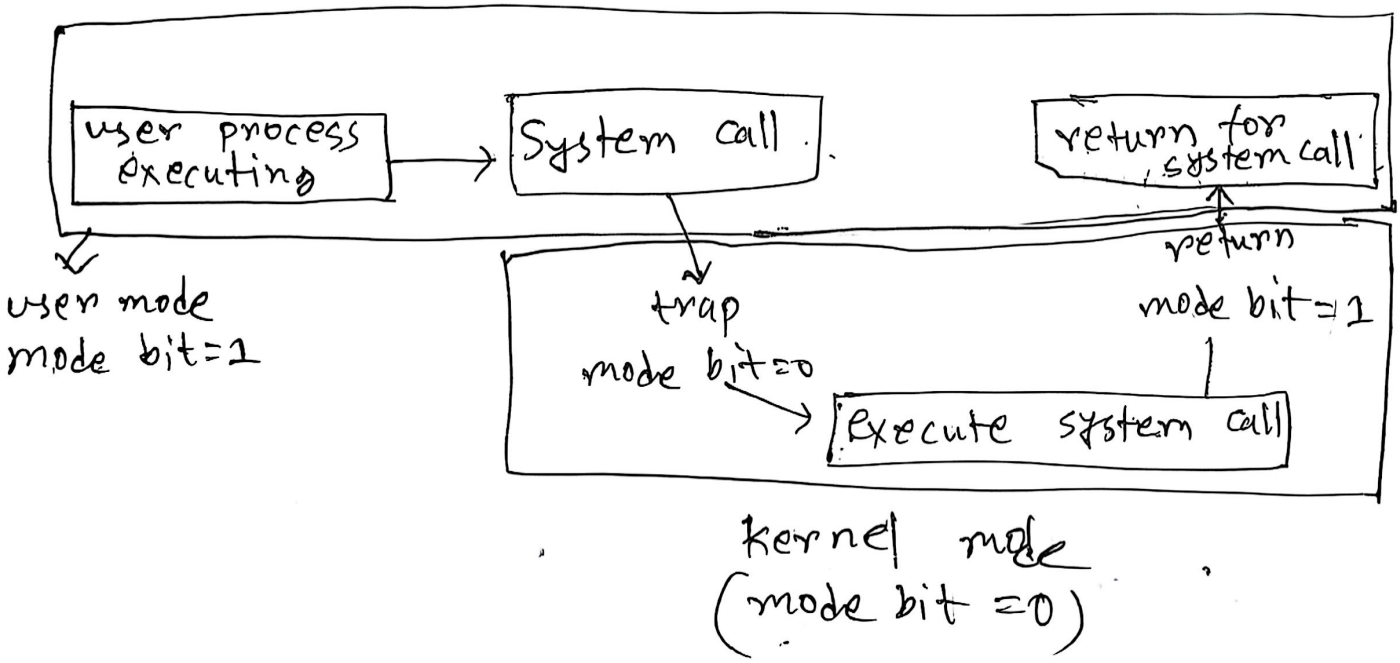
Purpose: Allows the operating system to protect itself and other running programs from bad user applications.

Hardware modes:

User mode (Mode bit=1): Executes standard user application code with restricted previlages

Kernel Mode (Mode bit=0): Executes core OS code with full access to hardware; previlaged instructions run only here.

Mode Switch: A user process makes a System Call;



slide page (41 + 45) [illegible]

Chapter-2

Lecture-5

02.08.2026

Operating-System Structes

An Operating system provides an environment for executing programs and offering services to users, processes and other systems. These features are split into two groups: those that help user to do their work, and those that keep the computer running safely and fast.

Operating System Services

This services are two types:

1. User-Focused Services: These functions designed to help the user perform tasks easily and execute applications. Such as:

(i) User Interface (UI): Almost all operating systems feature a UI to allow interaction.

↳ Types → Command-Line Interface (CLI)
→ Graphical User Interface (GUI)
→ Batch Interface

(ii) Program Execution: Loads a program into memory, runs it and ends its execution normally or with an error

(iii) I/O Operations: Handles input output requests required by running programms involving I/O devices

(iv) File-System Manipulation: Allows programs to read, write create, delete and search files or directories. Also handles file permissions. ~~and~~

(v) Communication: Enables processes to exchange information on the same computer or across network-connected computer

Methods: Shared memory or message passing.

(vi) Error detection: Continuously monitors the system for potential errors to ensure correct and consistent computing.

Location: CPU, main memory, I/O devices or user program.

Features: Includes debugging facilities to ~~assist~~ assist users and programs.

(2) System-focused Services: These services operate in the background to ensure resource sharing, efficiency and system security when multiple users run ~~seria~~ at the same time. Such as:

(i) Resource Allocation: Allocates system resources among multiple users or ~~simul~~ parallel tasks.

↳ Special allocation code: Used for CPU cycles, main memory and file storage.

↳ Release Code: User for I/O devices.

(ii) Accounting: Tracks which users use how many and what kinds of computer resources.

(iii) Protection and Security:

Protection: Controls access to system resources and prevents parallel processes from interfering with each other.

Security: Authenticates user to defend the systems and external I/O devices against unauthorized access from outsiders.

System Calls

Definition: A System Call is a programming interface that provides user-level processes a way to request OS services

Mechanism: It is an explicit request to the OS kernel made via a software interrupt from user processes.

Implementation: System calls are typically written in high-level languages like C, C++.

Application Programming Interface (API)

Definition: An API is a set of functions, objects, protocols or data structures that support application development for programmers.

Role as a Library or Layer: APIs sit between regular programs and the rest of the OS.

Fuction: APIs handle the low-level mechanis of passing parameters/information to the kernel and switching execution to supervisor mode.

Bridging Code: APIs define how services are requested in high-level programming to invoke system calls down in the low-level architecture.

Common APIs

⇒ Win32 API (for Windows)

⇒ POSIX API (for POSIX-based systems, including Linux, UNIX and macOS X).

⇒ Java API (for the Java Virtual Machine).
JVM

POSIX API

Definition: POSIX (Portable Operating System Interface) is a standardized set of Application Programming Interfaces (APIs) designed to maintain software

compatibility between different operating Systems.

Supported Systems: It is used across POSIX-based operating systems, including virtually all versions of UNIX, Linux and macOS.

Purpose: By holding to POSIX standards, developers can write application code in high-level languages (like C or C++) that can run across multiple Unix-like operating systems without ~~re~~ needing system-specific rewrites.

Functionality: It acts as bridge, wrapping underlying low-level system call sequences (such as process control, file manupulation and communication) into standardized function calls for programmers.

How System Calls are Implemented

System Call Numbering: Each system call usually has a unique number assigned to it.

System Call Table: The system call interface maintains an indexed table of these numbers to map API calls directly to their corresponding kernel function.

Execution Flow:

① A user application calls an API function in user mode.

② The system call interface intercepts it, looks up the system call number and switches to kernel mode.

③ The Kernel executes the actual system call implementation.

④ The system call interface returns the operation status and any return values back to user mode.

Abstraction: The developer doesn't need to know how the system call is executed under the hood. They only need to follow the API.

Example: To copy contents from a source file to a destinition file the below sequence is followed:

1. Acquire Input File Name: Display a prompt on the screen and read user input.

2. Acquire Output File Name: Display a prompt on the screen and read user input.

3. Open Input File: Abort if the file doesn't exist.

4. Create Output File: Abort if the file already exist

5. Copy Loop: Read data from the input file and write to the output file until read fails.

6. Clean Up & Exit: Close both files, print a completion message to the screen and terminate.

Operating System Design and Implementation

Designing and Implementing an operating system is a complex task with no single "correct" approach. However, it relies on fundamental goals, key design principles and strategic implementation choices.

1. Design Goals: Design Goals are defined by the intended audience and target hardware, split into two main catagories:

⇒ User Goals: → convinient, easy to use and easy to learn.
→ Reliable, safe and fast.

⇒ System Goals: → Easy to design, implement and maintain
→ Flexible, reliable, error-free and efficient

2. Mechanisms vs. Policies: A fundamental principle in software engineering and OS design is the separation of policy from mechanism:

Mechanisms: Determine how to do something.

Example: The hardware or low-level timer routine that performs a context switch.

Policies: Determine what will be done

Example: Deciding which process gets the CPU next and for how long.

3. Implementation: Operating Systems have evolved in terms of the programming languages used to write them:

Early OS: Written entirely in Assembly Language

- Adv → faster due to directly hard-coding
- Disd. → More no. of lines with large program

Modern OS: Written mostly in Algorithmic Languages (like C or C++), with a small amount of use of assembly language for critical routine.

Advantages:

(i) Code is written faster
(ii) Easier to read, debug and maintain.
(iii) More Compact.
(iv) Portability

Disadvantages:

(i) Reduced execution speed and higher memory usage.

Operating System Structure

Lecture-6

04.08.2026

structure

An Operating System refers to how the OS is organized internally. Different structures are designed to meet system goals such as high performance, modularity, security, maintainability and portability. Types.

1. Simple Structure: Early, small operating systems designed with minimal structure due to hardware limitations.

→Key Example: MS-DOS { written to provide the most functionality in the least space }

⇒Characteristics: i) Not divided into distinct modules.

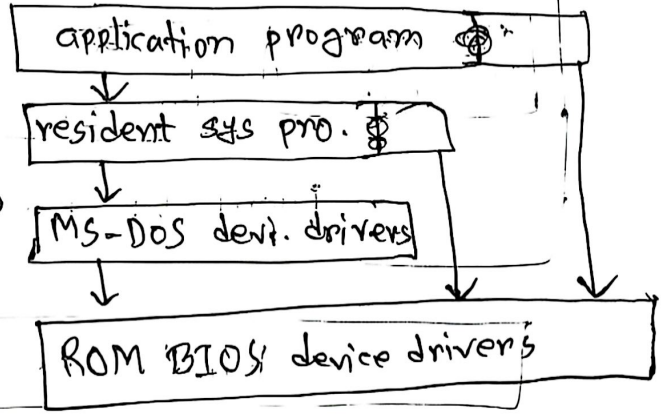
ii) Lacks dual-mode operation and hardware protection

iii) Difficult to extend; adding a new service requires modifying the entire OS.

iv) Highly vulnerable to malicious user program.

v) If a user program fails entire system crashes.

MS-DOS Layer structure →



2. Monolithic Structure: The entire operating system (file system, memory management, process scheduling, drivers) runs as a single, massive program in Kernel Mode.

Key Example: Early UNIX, traditional Linux kernel.

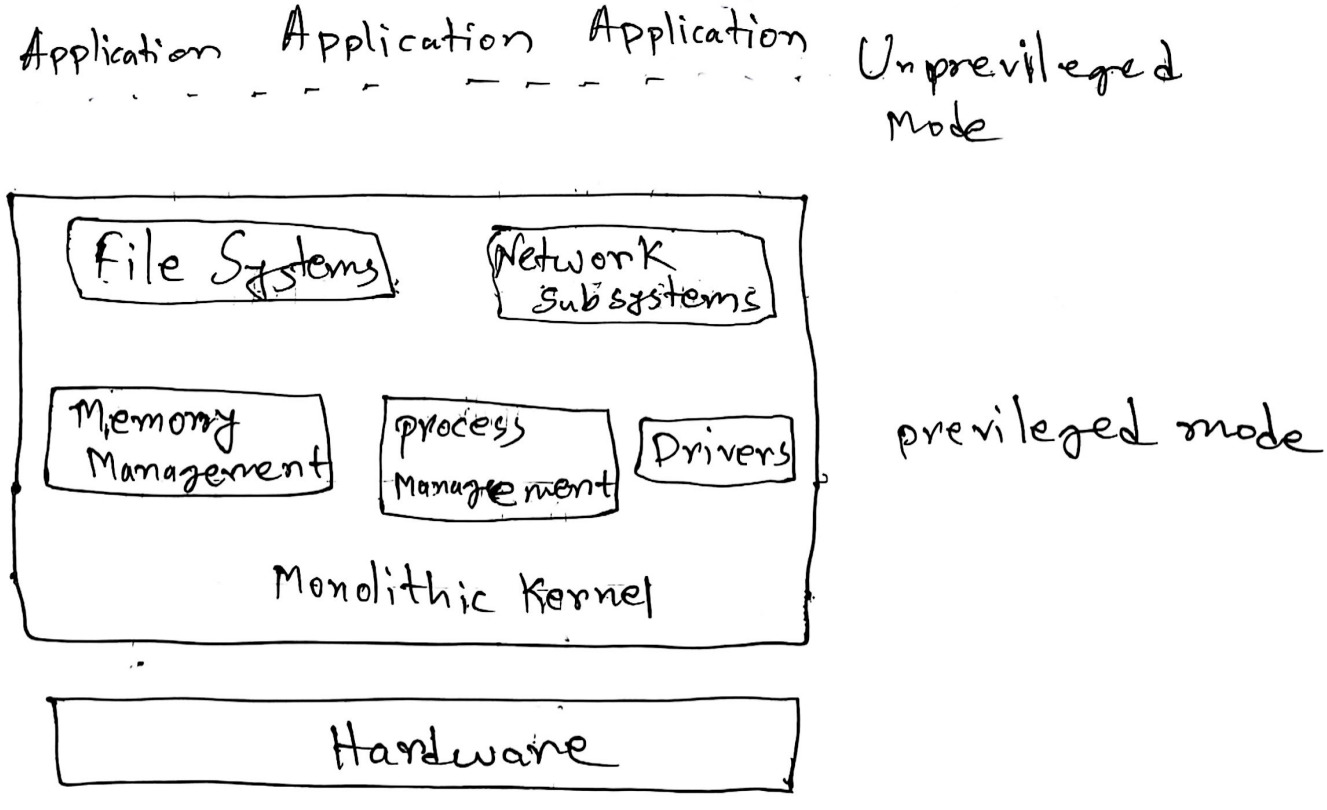
Pros: i) High Performance

ii) Fast Communication among kernel modules.

iii) Efficient system call execution.

Cons: (i) Large kernel size (ii) Difficult debugging.

(iii) A faculty driver may crash the entire OS.



3. Layer Structure: The OS is broken down into smaller, modular layers (level 0 to Level N). Each layer is selected such that it uses functions and services of only lower-level layers. Here,

Layer 0 ⟹ Hardware

Layer N ⟹ User Interface

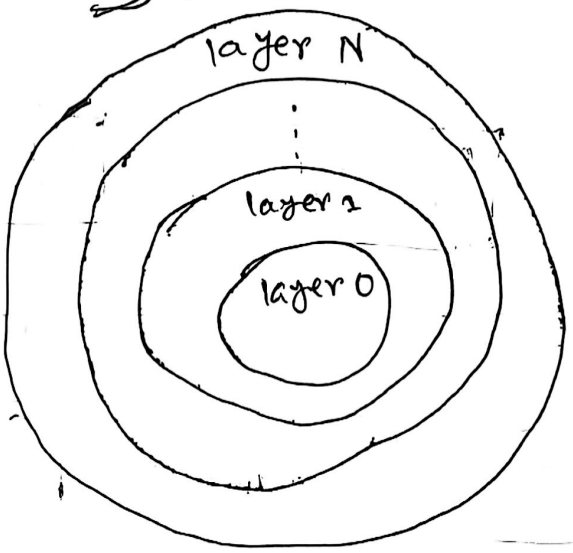
Pros: i) Simplicity of construction and debugging.

ii) Design and Implementation are simplified because lower layers are verified first.

iii) Gives More freedom in changing the inner workings of the system.

Cons: ① Major difficulty in appropriately defining each layer

ii) Less efficient than other structures: an I/O request from a user program adds processing overhead at each traversed layer. (বুঝার জন্য)



4. Microkernel Structure:

Microkernel system places only bare minimum system components in the kernel and moves all non-essential components out of the kernel into user space.

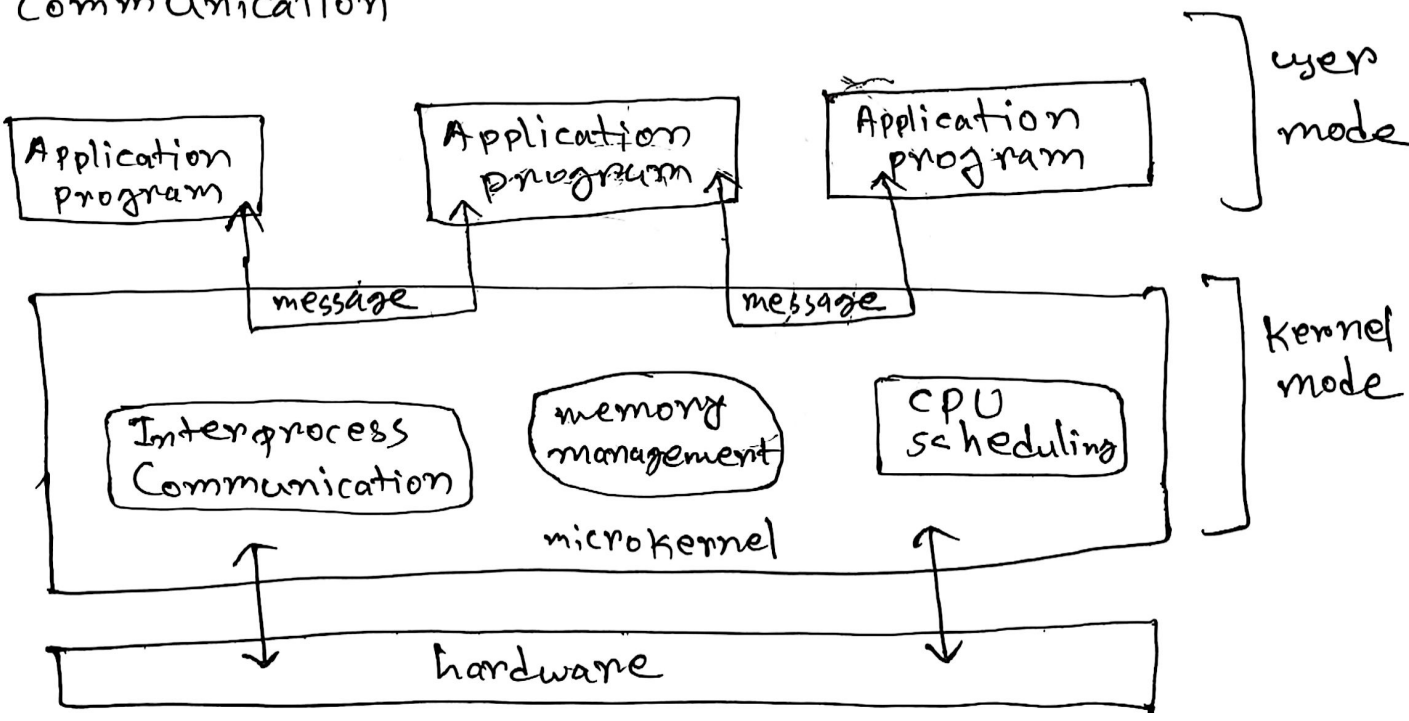
→ Functions like file systems, network protocols stacks, and device drivers run as user-mode process.

→ Modules communicate with each other using message passing through the microkernel.

Examples: Mach, Minix3, QNX.

Pros: (i) Easier to extend the microkernel
(ii) Easier to port the OS to new hardware architecture
(iii) More reliable (iv) Highly Secure.

Cons: (i) Performance overhead of user space to kernel space communication



5. Modular Architectures: (LKMs) Loadable Kernel Modules

The best current methodology for OS design is loadable kernel modules where the kernel has a set of core components and links in additional services via modules either boot time or during run time

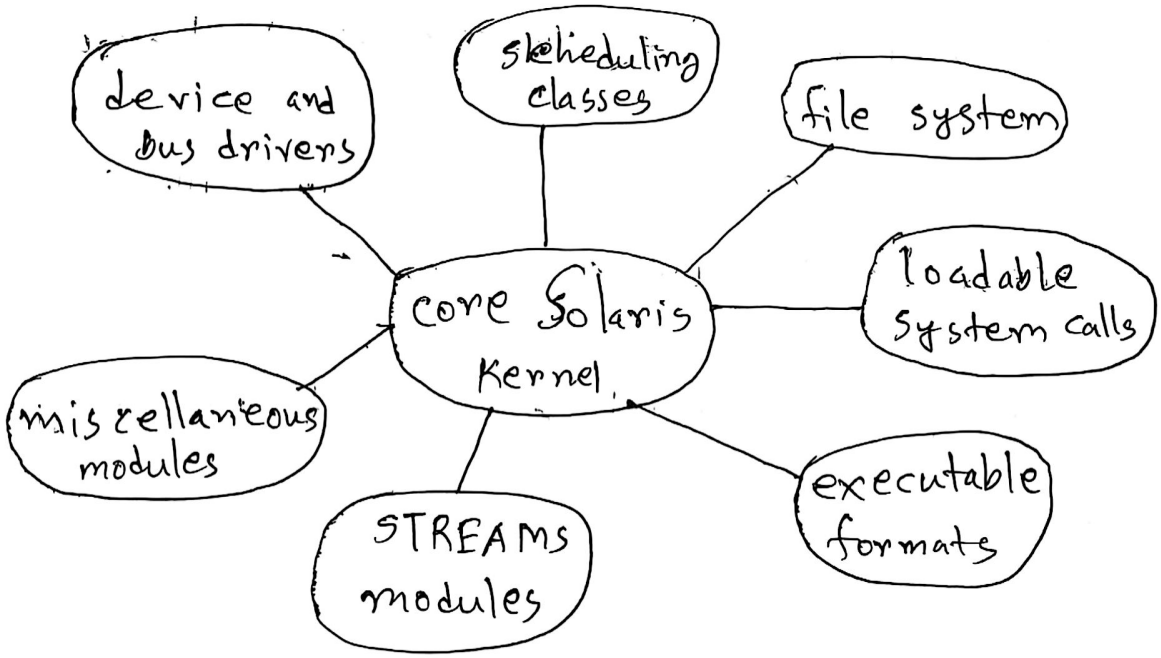
Avoids compiling the entire kernel every time when new features or drivers are added

Example: Solaris, Linux, macOS X

Pros: (i) More flexiable than a layer approach, (any modules can directly call another module).

ii) More efficient than microkernel because module do not require message passing to communicate.

Cons: Memory fragmentation penalty; inserting new kernel module code dynamically can fragment kernel memory and impact Translation Lookaside Buffer (TLB) usage.



6. Hybrid Structure: Combines multiple structural ~~struc~~ approaches to address performance, security and usability needs simultaneously. Most ~~M~~ modern OSs are not purely one model.

→ Linux/Solaris: Monolithic core inside kernel address space for speed, paired with modular for dynamic loadi

→ Windows: Mostly monolithic for performance plus microkernel-style concepts to support different user-mode subsystems.

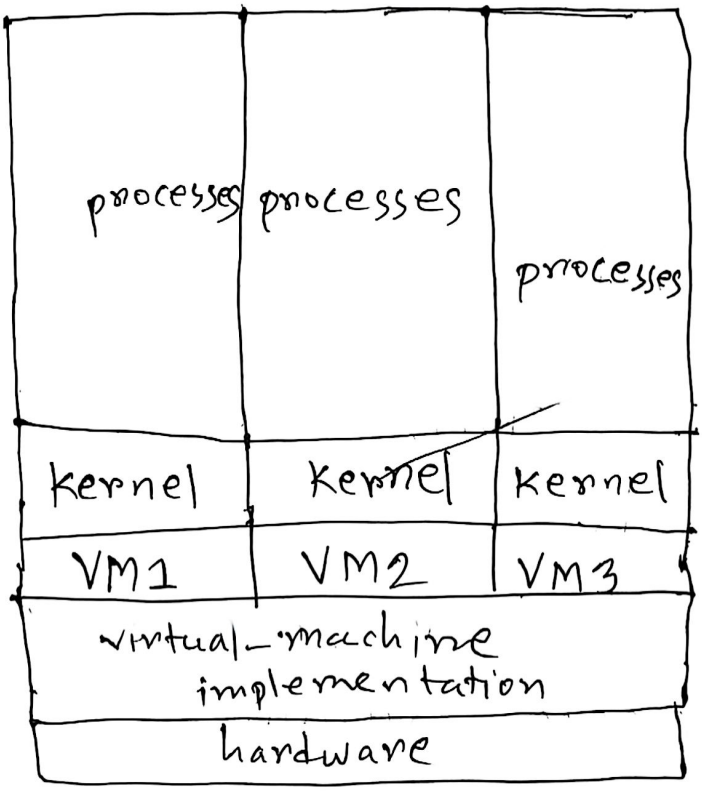
Example: Apple macOS (Mac OS X, ios Android, M. Windows.

7. Virtual Machine: Takes the layered approach to its logical conclusion by ~~treating~~ both physical hardware and the OS kernel as if they were entirely hardware.

→ The operating system host creates ~~the~~ illusion that a process has its own processor and virtual memory.

→ Each guest is provided with a copy of underlying computer.

virtual machine



8 Exokernel Structure: Provides only minimal abstraction over hardware, granting applications direct access to physical hardware resources.

→ Allows applications to manage resources directly and make decicions previously reserved for the kernel.

→ Applications manage raw resources through Library Operating Systems [LibOS). [Example→ MIT Exokernel, Aegis]

Stack: Applications → LibOS → Exokernel → Hardware.

Pros:
(i) Maximize system performance
(ii) Highly customizable
(iii) Extremly efficient resource utilization

Cons:
(i) Difficult to program
(ii) Rarely used in Commercial systems

Chapter-3: Process

Lecture-7

11.08.2026

Definition: A process is an active program currently in execution, unlike a passive program stored on a disk.

⇒ The same program can have multiple active processes running at once.

Memory Layout / Process Section

Text Selection: Contains the executable instructions or code.

Data Section: Stores global variables (split into initialized and uninitialized data).

Heap: Memory dynamically allocated at runtime. (using malloc()).

Stack: Stores temporary execution data such as

function parameters, return addresses and local variables

High memory: Holds commandline arguments (argc, argv) passed to main() function.

{ figure: slide page- 2, 3

Process states and State Transitions

New: The process is being created

Ready
~~Running~~: The process is waiting to be assigned to a CPU core.

Running: Instructions are actively executing on the CPU.

Waiting: The process is blocked, waiting for an I/O event or signal.

Terminated: The process has completed its execution

Transitions:

- New → Ready (Admitted)
- Ready → Running (Sheduler dispatch)
- Running → Ready (Interupt)
- Waiting → Ready (I/O or event completion)

Running → Terminated (Exit)

slide page-6 ↑ (for figure)

Process Control Block (PCB)

The PCB (also called a Task Control Block) maintains all information needed to represent, run or suspend a process.

- Process State: Current state (New, Ready, Running, Waiting, etc).
- Program Counter (PC): Address of the next instruction to execute.

CPU Registers: State registers (stack pointers, index registers) that must be saved upon an interrupt to safely resume execution later.

CPU Scheduling Information: Priorities, queue pointers and scheduling parameters.

Memory-Management Information: Base/limit registers, page tables or segment tables.

Accounting Information: CPU execution time, time limits, process ID numbers and accounts.

I/O Status Information: Allocated I/O devices, open file descriptors and buffers.

Lecture-8
18.08.2026

Process vs Thread

follow. slide-30 page (9, 10)

Process Scheduling

Process Scheduling is the Method an operating system uses to decide which process gets to run when multiple process are ready the process scheduler selects a process from the ready queue and allocates CPU time to it.

. It maintains scheduling queues of processes.

Scheduling Queues

As processes enter and transition through the system, their Process Control Blocks are maintained in several queues:

Job Queue: Holds all processes in the system, stored on secondary storage (disk) and managed by the long-term scheduler.

Ready Queue: Holds processes that reside in the main memory and are in the ready state waiting to be allocated CPU time. It is managed by the short-term scheduler.

Wait/Device Queue: Holds processes waiting for specific I/O devices or events e.g. disk I/O, child process exit or interrupt completion.

Scheduling Queues (representation)

Once dispatched from the ready queue to the CPU a process follows one of several execution paths:

1. **I/O Request:** The process issues an I/O request and is placed in an I/O wait queue; upon I/O completion, it returns to the ready queue.

2. **Child Process Creation:** The process forks a child process and waits in a wait queue until the child terminates, after which it returns to the ready queue.

3. **Interrupt / Time Slice Expiry:** The process is forcely removed from the CPU core due to a hardware interrupt or timer expiration and placed directly back into ready queue.

4. **Termination:** Upon completing its task, the process terminates and is removed from all queues; its PCB and resources are deallocated.

Types of Schedulers

i) Short Term Scheduler: (CPU scheduler): selects which process from the ready queue in memory should execute next on the CPU. This is invoked very frequently (in milliseconds). and must be fast to minimize overhead.

ii) Long-Term Schedulers: (Job Schedulers): Selects which processes from the secondary storage job queue should be brought into main memory (the ready queue). This is Invoked infrequently (seconds or minutes) and control the degree of multiprogramming (the number of processes in memory).

Context Switching

Context Switching is the operation by which an operating system saves the execution state (context) of a currently running process or thread and restores the saved state of another process or thread so the CPU can switch execution beetwen them.

fig (slide-3 page-16)

Operations on Processes

Lecture - 9

23.08.2026

The processes in a system execute concurrently and can be created and deleted dynamically. OS must provide mechanisms for both Process and Creation and Process Termination

Process Creation:

⇒ **Process Tree:** A creating process is called the parent and newly created processes are called children. Children can create further processes, forming a tree hierarchy. (e.g. in Linux, rooted at init with PID = 1).

⇒ **Process Identifier (PID):** Most operating systems identify and manage processes using a unique integer PID.

⇒ **Resource Sharing Options:**

→ Parent and children share all resources.
→ Children share a subset of the parent's resources.
→ Parent and children share no resources.

⇒ **Execution Options:** → The parent continues to execute concurrently with its children.
→ The parent waits until some or all children terminates.

⇛ Address Space Options:

Duplicate: The child process is an exact copy of the parent

New Program: The child process has a completely new program loaded into its address space.

Implementation Examples

UNIX/LINUX

→ fork(): System call that ~~includes~~ duplicates the pa process into a new child process. It returns 0 to the child and the child's PID (>0) to the parent.

→ execl()/execlp(): System call invoked after fork() to overwrite the child's address space with a new binary program.

→ wait(): The parent invokes this to suspend execution untill the child terminates.

2. Process Termination

→ Normal Termination. A process executes its final statement and calls exit() to request deletion.

- The process returns exit status data to its parent (via wait()).
- The OS dellocates and reclaims all physical/virtual memory, open files and I/O buffers.

→ Abnormal/Parent Termination: A parent can terminate a child process (using abort()) in UNIX due to:

- → Child exceeding allocated resources
- → Task assigned to the child is no longer needed.
- → The parent is exiting and the OS does not permit child processes to run without a parent.

Special Terminations

→ Cascading Termination: When an operating system automatically terminates all child, grandchild and descendant processes if their parent terminates.

→ Zombie Process: A process that has finished execution but those parent has not yet called wait() to collect its exit status and free its PCB entry.

→ Orphan Process: A process whose parent has terminated without calling wait(). These processes are adopted by a root process (init or systemd), which periodically calls wait() to clean them up.

Interprocess Communication

Interprocess Communication (IPC) is a set of mechanisms provided by the operating system that allows processes to exchange data and synchronize their actions.

Process Types:

Independent Process: A process is independent if it does not share data with any other process and cannot affect or be affected by the execution of another process.

Cooperating Process: A process is cooperating if it can

affect or be affected by other processes executing in the system.

Reasons for Cooperating Processes

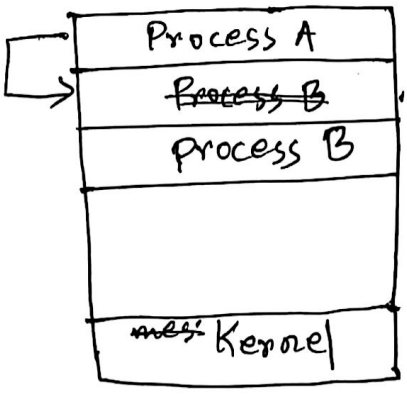
→ Information Sharing: Multiple processes may need access to the same piece of information.

→ Compution Speedup: A particular task can be broken into subtasks that run concurrently across multiple processing cores to execute faster.

→ Modularity: Constructing a system in a modular fashi by dividing system functions into separate processes or thread

Models of IPC

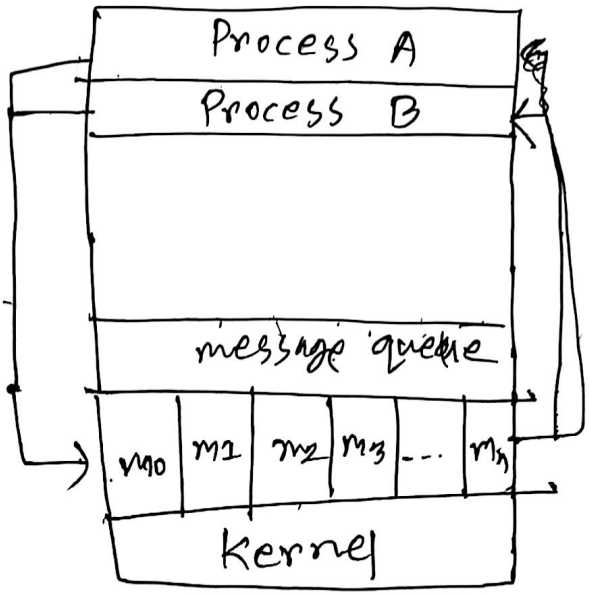
1. Shared-Memory: A region of memory shared by cooperating processes is established. The processes exchange information by directly reading and writing data in this shared memory area. Main advantage is faster communication at memory-transfer speeds.



shared memory

2. Message-Passing Model: Communication occurs by exchanging predefined formatted messages between cooperating ~~systems~~ processes (using system operations such as send(message) and receive(message).

Slower than shared memory due to system call overhead and kernel intervention. However it avoids memory access conflicts.



Chapter-5 (CPU Scheduling)

CPU - I/O Burst Cycle:

→ CPU Burst: A period of time when the process is actively executing instructions on the CPU. (such as arithmetic operations, loads and stores).

→ I/O Burst: A period of time when the process is idle with respect to the processor, waiting for an input/output transfer to complete. (such as disk or network operation).

Process execution is fundamentally characterized by an alternating cycle of CPU execution and I/O waiting. A process begins with a CPU burst, followed by an I/O burst, which is followed by another CPU burst, repeating continuously throughout its lifecycle. The execution sequence terminates on a final CPU burst with a system request to end execution

CPU scheduling

Ready Queue: A data structure that keeps track of all processes residing in main memory that are ready and waiting to execute on a CPU core.

Process Control Block (PCB): A kernel data structure containing all information about a specific process (such as register state, program counter); elements in ~~sche~~ scheduling queues are typically PCBs.

Preemptive scheduling: A scheduling scheme where the operating system can interrupt and forcibly remove the CPU from an executing process before its burst ends.

Non-Preemptive scheduling: A scheduling scheme where a process retains the CPU until it voluntarily control, either by termination or by transitioning to a waiti state.

Working Principle: Whenever the CPU becomes idle, the CPU scheduler (short-term scheduler) selects an eligible process from the ready queue in memory and allocates the CPU core to it. The ready queue is not necessarily a FIFO queue; It can be implemented as a FIFO queue, priority queue, balanced tree or an unordered linked list.

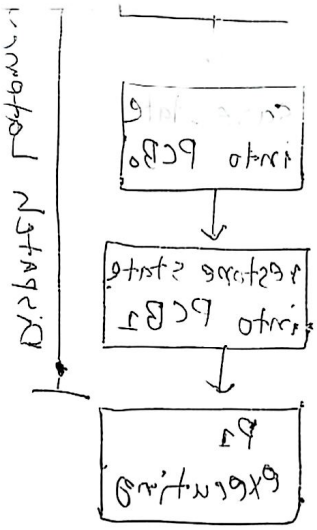
Scheduling decision occurs under four situation.

i) A process switches from running to waiting (e.g I/O request).

ii) A process switches from running to ready. (e.g. interrupt occurs).

iii) A process switches from waiting to ready.

iv) A process terminates.



Dispatcher

Lecture-10

30.08.2026

~~Cont~~ While the CPU scheduler is responsible for deciding which process gets the CPU, the dispatcher is the module that actually gives control of the CPU core to that selected process. To hand over control, the dispatcher performs three specific actions:

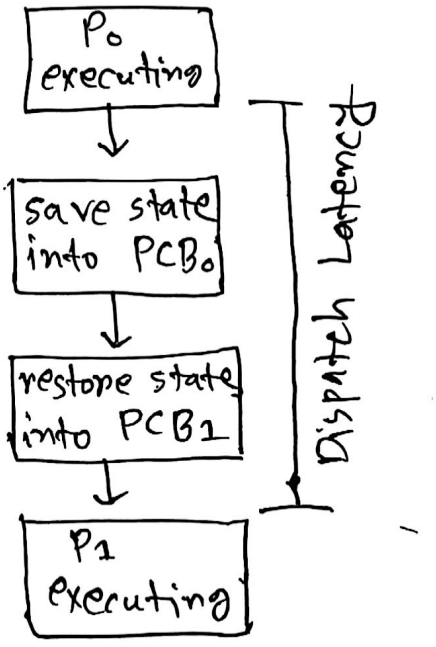
→ Context switching from one process to another.

→ Switching to User mode.

→ Jumping to proper location in the user program to resume execution.

Role: The dispatcher is the module that actually gives control of the CPU core to the process selected by the CPU scheduler.

Dispatch Latency: The time required for the dispatcher to stop one process and start another running. Because it is invoked on every context switch, it must be as fast as possible.



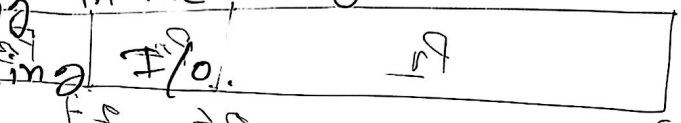
CPU - Scheduling Criteria

Different CPU-scheduling algorithms ~~processess~~ possess different properties and may favor one class of processes over another. To evaluate and compare scheduling algorithms several performance and metrics are used:

→ CPU Utilization: The percentage of time the CPU is actively executing process. While conceptually ranging from 1% to 100%, real-world systems typically operate between 40% to 90%.

→ Throughtput: The number of processes completed per unit of time. For long-running processes, this may be one process every few seconds; for short tasks, it can reach tens of processes per second.

→ Turnaround Time: The total time elapsed from the submission of a process to its completion. It equals the sum of time spent waiting in the ready queue, executing on the CPU, and performing I/O.



→ Waiting Time: The cumulative amount of time a process spends waiting in the ready queue. The CPU scheduler does not alter the execution or I/O time of a process.

→ Response Time: The duration from request submission untill the first response is produced. This metric is critical in interactive/time-sharing environments because users care about how quickly execution begins rather than total completion time.

Extra

5.3 Scheduling Algorithms

FCFS (First Come First Served)

Case-1: Arrival time 0

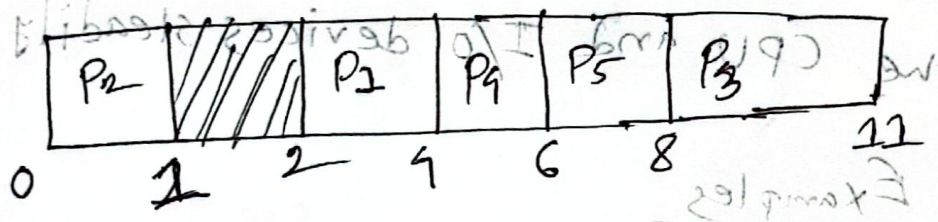
Process	Burst Time
P_1	24
P_2	3
P_3	3

Average Waiting Time (AWT) = (0+24+27)/3

= 51/3 = 17

Case-2: Different Arrival time

Process	Arrival Time	Burst Time
P_1	2	1
P_2	0	3
P_3	5	2
P_4	3	2
P_5	4	1



$$AWT = \{(2-2) + (0-0) + (8-5) + (4-3) + (6-4)\}$$

$$= (0 + 0 + 3 + 1 + 2)/5$$

$$= 1.2$$

Lecture- 11

01.09.2026

SJF (Shortest Job First)

Why SJF is Better than FCFS?

Provably Minimal Average Waiting Time: SJF gives the smallest possible average waiting time compared to any other scheduling method.

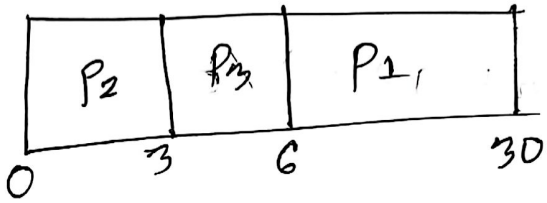
Prioritizing Shorter Jobs: Serving quick tasks first helps everyone finish sooner on average. If a heavy task run first, all small task get stuck waiting behind it, which ruins average wait times.

Elimination of Convoy Effect: FCFS causes a bottleneck (the convoy effect) where many quick tasks wait idly in line while one long task hogs the CPU. SJF runs the quick tasks right away, keeping both the CPU and I/O devices steadily busy

Examples

Case-1:

$P_1 - 24$
$P_2 - 3$
$P_3 - 3$

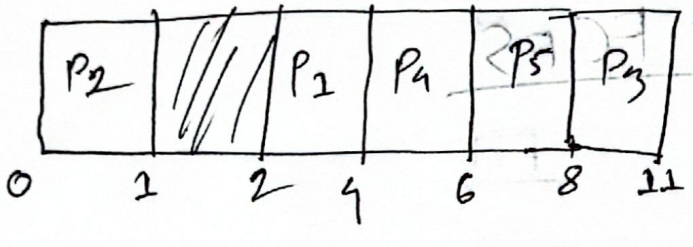


$$AWT = (6 + 0 + 3)/3$$

$$= 3$$

Case-2

Process	AT	BT
P_1	2	2
P_2	0	1
P_3	5	3
P_4	3	2
P_5	4	4



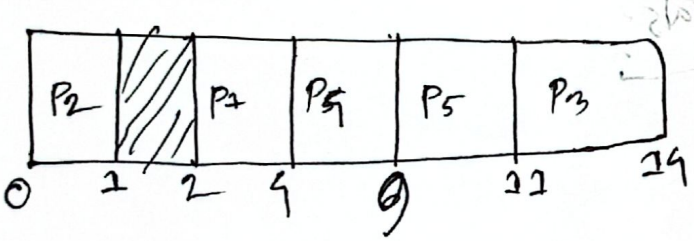
$$AWT = \{(0-0)+(2-2)+(9-3)+(6-4)+(8-5)\}$$

$$= (1+2+3)/5$$

$$= 1.2$$

Example-2

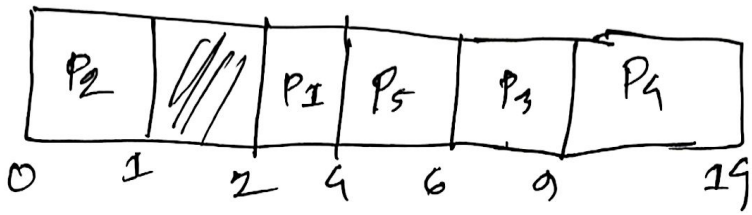
Process	AT	BT
P_1	2	2
P_2	0	1
P_3	5	3
P_4	3	5
P_5	4	2



$$AWT = \{(0-0)+(2-2)+(9-3)+(9-4)+(11-5)\}/5$$

$$= 2.4$$

SJF



$$AWT = \{(0-0) + (2-2) + (4-4) + (6-5) + (9-3)\} / 5$$

$$= 7/5 = 1.4$$

SJF > FCFS

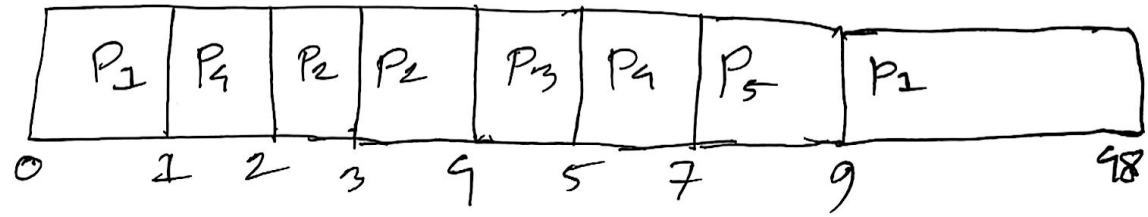
SRJF

Why SRJF (Preemptive SJF) is better than (Non preemtive SJF) ?

Dynamic Adaption to Late Arrivals:

Example

P	AT	BT	RT
P_1	0	40	40
P_2	2	2	2
P_3	3	1	1
P_4	1	3	3
P_5	4	2	2



$WT_1 = 48 - 0 - 40 = 8$

$WT_2 = 4 - 2 - 2 = 0$

$WT_3 = 5 - 3 - 1 = 1$

$WT_4 = 7 - 1 - 3 = 3$

$WT_5 = 9 - 4 - 2 = 3$

$AWT = (8 + 0 + 1 + 3 + 3) / 5$

$= 3.4$

Priority Scheduling

Lecture – 12
6.9.2026

Priority Scheduling
- Preemptive
- Non-preemptive

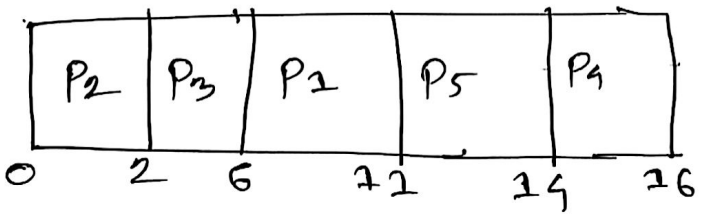
Non-preemptive: এখানে priority value দেওয়া থাকবে

Low integer = high priority

$P_1 = 1$
$P_2 = 5$
} P_1 এর (1) এই কেন priority আগে

non-preemptive - এ interrupt করা যাবে না।

P	AT	BT	Priority
P_1	3	5	2
P_2	0	2	3
P_3	2	4	1
P_4	5	2	6
P_5	10	3	5



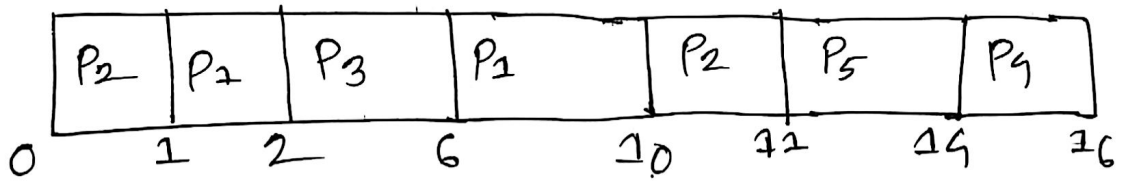
$$AWT = \frac{(6-3)+(0-0)+(2-2)+(4-5)+(11-10)}{5}$$

$$\boxed{AWT = 2.6}$$

Preemptive: Emergency / high priority execute আগে করতে হয়।

আগের Table অনুযায়ী নাও।

Process	AT	BT	Priority
P_1	1	5	2
P_2	0	2	3
P_3	2	4	1
P_4	5	2	6
P_5	10	3	5



waiting time,

$P_1 = 10 - 1 - 5 = 4$

$P_2 = 11 - 0 - 2 = 9$

$P_3 = 6 - 2 - 4 = 0$

$P_4 = 16 - 5 - 2 = 9$

$P_5 = 14 - 10 - 3 = 1$

$$AWT = \frac{4+9+0+9+1}{5}$$

$$= 24.6$$

Round Robin

A major problem with priority scheduling is that low priority processes can remain blocked indefinitely in heavily loaded systems where a continuous stream of higher-priority processes keeps arriving. This situation is called starvation.

To solve this,
A primary solution to starvation that gradually increases the priority of processes waiting in the system over time. Eventually, even the lowest-priority process advances to a high enough priority to be dispatched.

Round Robin

Same as time sharing system

- Time Quantum (समय खंड) (TQ)
- by nature preemptive

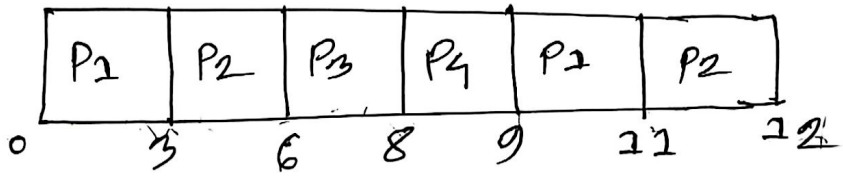
Ready Queue: Sequence में रखना आसान।

P_1	AT	BT	RT
P_1	0	5	2
P_2	1	4	1
P_3	2	2	0
P_4	3	1	0

Data

TQ = 3

Gantt chart →



Answer লিখার সময়ে Gantt chart দেখাতে হবে আর বুঝার সুবিধার্থে কোন কোন answer করতে হবে।